

Exercícios — Aula 18 — Combinações, subconjuntos e permutações

Técnicas de Programação - 2026/02

Última atualização: October 5, 2026

Instruções

- Diferencie a pergunta: qualquer subconjunto, subconjunto de tamanho fixo, combinação ou permutação.
- Use `inicio` quando a ordem não importa e não queremos duplicar combinações.
- Use `usado[]` quando a ordem importa e cada elemento pode ocupar posições diferentes.

Exercício 1 — Simulação de permutações com `usado[]`

Simule o algoritmo para $v = [1, 2, 3]$.

Algorithm 1: Permutacoes

Result: Executa PERMUTACOES.

Input : array v

Output: list respostas

```
respostas ← LISTA-VAZIA()
atual ← LISTA-VAZIA()
usado ← ARRAY-DE-FALSOS(TAMANHO(v))
PERMUTAR(v, usado, atual, respostas)
return respostas
```

Algorithm 2: Permutar

Result: Executa PERMUTAR.

Input : array v , array usado, list atual, list respostas

Output: none

```
if TAMANHO(atual) = TAMANHO(v) then
    ADICIONAR-COPIA(respostas, atual)
    return
end
for  $i \leftarrow 0$  to TAMANHO(v) - 1 do
    if usado[i] = false then
        usado[i] ← true
        ADICIONAR(atual, v[i])
        PERMUTAR(v, usado, atual, respostas)
        REMOVER-ULTIMO(atual)
        usado[i] ← false
    end
end
```

Preencha a tabela para as primeiras chamadas até registrar $[1, 2, 3]$, depois continue listando todas as respostas.

	passo	atual antes da escolha	usado antes	escolha	resposta registrada?
	1	<code>[]</code>	<code>[F, F, F]</code>		
	2				
	3				
	4				

Liste todas as permutações registradas na ordem do algoritmo.

Exercício 2 — Combinações de tamanho k

Escreva pseudocódigo para gerar combinações de tamanho k .

Contrato:

- entrada: array v , inteiro k ;
- saída: lista de combinações com exatamente k elementos;
- use parâmetro `inicio`;
- registre uma cópia quando `TAMANHO(atual) = k`;
- depois de escolher um elemento, a próxima chamada deve começar em $i + 1$.

Teste com:

- $v = [1, 2, 3, 4]$, $k = 2$;
- $v = [1, 2, 3]$, $k = 0$;
- $v = [1, 2]$, $k = 3$.

Exercício 3 — Subconjuntos de tamanho k

Agora gere subconjuntos de tamanho k usando a lógica incluir/não incluir.

Complete o pseudocódigo.

Algorithm 3: SubconjuntosK

Result: Executa SUBCONJUNTOS-K.

Input : array v , int k

Output: list respostas

```
respostas  $\leftarrow$  LISTA-VAZIA()
BACKTRACK-K( $v, 0, k, \text{LISTA-VAZIA}(), \text{respostas}$ )
return respostas
```

Algorithm 4: BacktrackK

Result: Executa BACKTRACK-K.

Input : array v , int i , int k , list atual, list respostas

Output: none

```
if TAMANHO(atual) >  $k$  then
    return
end
if  $i = \text{TAMANHO}(v)$  then
    if _____ then
        _____
    end
    return
end
BACKTRACK-K( $v, i + 1, k, \text{atual}, \text{respostas}$ )
ADICIONAR(atual,  $v[i]$ )
BACKTRACK-K( $v, i + 1, k, \text{atual}, \text{respostas}$ )
```

Compare com o Exercício 2:

1. Qual versão tende a fazer chamadas que já sabemos que não completam tamanho k ?
2. Como poderíamos podar quando faltam poucos elementos?

Exercício 4 — Qual estado usar?

Escolha entre `i`, `inicio` e `usado[]`.

Tarefa	Estado mais adequado	Justificativa
Gerar todos os subconjuntos		
Gerar combinações de tamanho 3		
Gerar permutações		
Decidir incluir/não incluir cada item da mochila		
Escolher uma ordem de apresentação de grupos		
Escolher um grupo sem importar a ordem		

Depois responda:

1. Por que `inicio` evita combinações duplicadas?
2. Por que `usado[]` é necessário em permutações?
3. Em quais tarefas a ordem da resposta importa?

Exercício 5 — Duplicação de combinações

O algoritmo abaixo tenta gerar combinações de tamanho 2, mas produz duplicatas em ordens diferentes.

Algorithm 5: CombinacoesComErro

Result: Executa COMBINACOES-COM-ERRO.

Input : array v , int k , list $atual$, list $respostas$

Output: none

```
if TAMANHO( $atual$ ) =  $k$  then
    ADICIONAR-COPIA( $respostas, atual$ )
    return
end
for  $i \leftarrow 0$  to TAMANHO( $v$ ) - 1 do
    if  $v[i]$  not in  $atual$  then
        ADICIONAR( $atual, v[i]$ )
        COMBINACOES-COM-ERRO( $v, k, atual, respostas$ )
        REMOVER-ULTIMO( $atual$ )
    end
end
end
```

1. Simule para $v = [1, 2, 3]$, $k = 2$.
2. Quais respostas duplicadas aparecem como ordens diferentes?
3. Qual parâmetro deve ser adicionado para evitar isso?
4. Reescreva o laço usando esse parâmetro.

Exercício 6 — Quantidade de respostas

Complete a tabela.

n	subconjuntos 2^n	permutações $n!$	combinações de tamanho 2
1			
2			
3			
4			
5			
6			

Depois responda:

1. Qual cresce mais rápido: 2^n ou $n!$?
2. Por que permutação costuma explodir rapidamente?
3. Para $n = 10$, quantas permutações existem?
4. Por que é perigoso testar geração de todas as permutações apenas com entradas pequenas?

Créditos e reaproveitamento

Exercícios majoritariamente novos. Quando houver inspiração direta do acervo antigo de backtracking, árvore de decisões e enumeração exaustiva, indicar:

Adaptado de material de Igor Montagner para a disciplina Técnicas de Programação.